CSC123 - Programming and Problem Solving
Project 03: Composition vs. Inheritance

PART 1 — COMPOSITION ("has-a")

The four Java files provided (Point.java, Square.java, Cube.java, Driver.java) demonstrate COMPOSITION — a design pattern where objects are built by containing other objects as member variables.

- Point has-a three doubles (x, y, z coordinates)
- Square has-a Point (its corner location)
  has-a double (its side length)
- Cube has-a Square (one face of the cube)
  has-a double (its depth)

Each class CONTAINS an instance of another class. A Square object holds a Point object as a private member variable. A Cube object holds a Square object as a private member variable. This is called "has-a" because a Square literally has a Point inside it.

Step 1: Compile and run the provided files.
Step 2: Trace the output and confirm you understand how the constructors chain through Cube → Square → Point.

WHY NOT JUST COPY AND PASTE?

Before we introduce inheritance, it's worth asking: why not simply copy a working class and modify it for each new variation? This approach — sometimes called "copy-paste reuse" — seems convenient at first, but creates two serious problems as a codebase grows:

1) Organizational chaos
   As variations accumulate, you end up with dozens of slightly modified source files derived from each other. When a bug is found, some versions need fixes, others don't. Keeping them consistent requires extreme discipline. Without it, the codebase devolves into duplication, divergence, and fragile maintenance.

2) Inconsistent fixes and high risk
   If you need a small change to an existing class, you edit the source directly. Even minor modifications can break hidden dependencies. To be safe, you must fully re-analyze the original logic — often nontrivial — just to ensure nothing else is damaged.

Copy-paste reuse scales linearly in files, but exponentially in maintenance cost.

Inheritance solves both problems. A fix made in a parent class is automatically inherited by all child classes. There is one source of truth, not many drifting copies. This is the core motivation for what you are about to build.

PART 2 — INHERITANCE ("is-a")

Your task is to re-implement the same three shapes using INHERITANCE instead of composition. Inheritance is a different way to build on existing classes: rather than containing another object, a class extends it and inherits all of its fields and behavior.

The shift in thinking:

Composition: A Square HAS-A Point (Square contains a Point variable)
Inheritance: A Square IS-A Point (Square extends Point directly)

With inheritance:

- Point extends Shape — a Point is-a Shape with x, y, z coordinates
- Square extends Point — a Square is-a Point with one double (side length)
- Cube extends Square — a Cube is-a Square with one double (depth)

Key rules for your implementation:

1. Your new Square class must NOT declare a Point as a member variable. It extends Point — Point's fields are inherited automatically.

2. Your new Cube class must NOT declare a Square or Point as a member variable. It extends Square — all fields from both Point and Square are inherited.

3. Use the provided Shape class as the root of the hierarchy. Each class should call super(...) in its constructor to initialize the parent class.

4. Override toString() and getName() appropriately at each level.

⚠ **Important:** You must submit *all required Java files*.
Incomplete submissions (missing classes) will receive significant deductions.

Required Files (All Mandatory)
You must submit **ALL** of the following .java files:

- Shape.java
- Point.java
- Square.java
- Cube.java
- Circle.java
- Cylinder.java

Submission Rules
• Each class must be in its own .java file
• File names must match class names exactly
• Submit **all files together**
• Projects missing files will not compile and will lose points
• Only .java files should be submitted
If one file is missing, the entire project cannot be properly graded.

REQUIRED: Comment Header Block (Mandatory)
Every file must begin with this exact header format.
Failure to include this header results in a deduction.

```
/**********************************************************************
 * Name: First Last
 * Course: CSC 123
 * Project: 03
 * Date:
 * File: Shape.java
 *
 * Description:
 * Brief description of what this class represents.
 **********************************************************************/
```

Header Requirements
• Must be at the very top of the file
• Must list the correct file name
• Must describe the class
• Must be properly formatted
• Every file must have its own correct header

Class Structure
You will build the following inheritance hierarchy:

Shape

Point

Square

Cube

Circle

Cylinder

Each class inherits from the one above it.

Part A — Shape (Base Class)

Field

private String name;

Constructor

public Shape(String name)

Methods

public String getName()

public void setName(String name)

public String toString()

toString() should return something similar to:

Shape: <name>

Part B — Point extends Shape

Fields

private double x;

private double y;

private double z;

Constructor

public Point(double x, double y, double z)

Must call:

super("Point");

Override toString() to include coordinates.

Example:

Point (x=1.0, y=2.0, z=3.0)

Part C — Square extends Point

Field

private double side;

Constructor

public Square(double side, double x, double y, double z)

Must call:

super(x, y, z);

Method

public double area()

Return:

side * side

Override toString().

Part D — Cube extends Square

Field

private double depth;

Constructor

public Cube(double depth, double side, double x, double y, double z)

Must call:

super(side, x, y, z);

Method

public double volume()

Return:

side * side * depth

Override toString().

Part E — Circle extends Point

Field

private double radius;

Constructor

public Circle(double radius, double x, double y, double z)

Must call:

super(x, y, z);

Method

public double area()

Return:

Math.PI * radius * radius

Override toString().

Part F — Cylinder extends Circle

Field

private double height;

Constructor

public Cylinder(double height, double radius, double x, double y, double z)

Must call:

super(radius, x, y, z);

Method

public double volume()

Return:

Math.PI * getRadius() * getRadius() * height

Override toString().

Driver.java Requirements (provide)

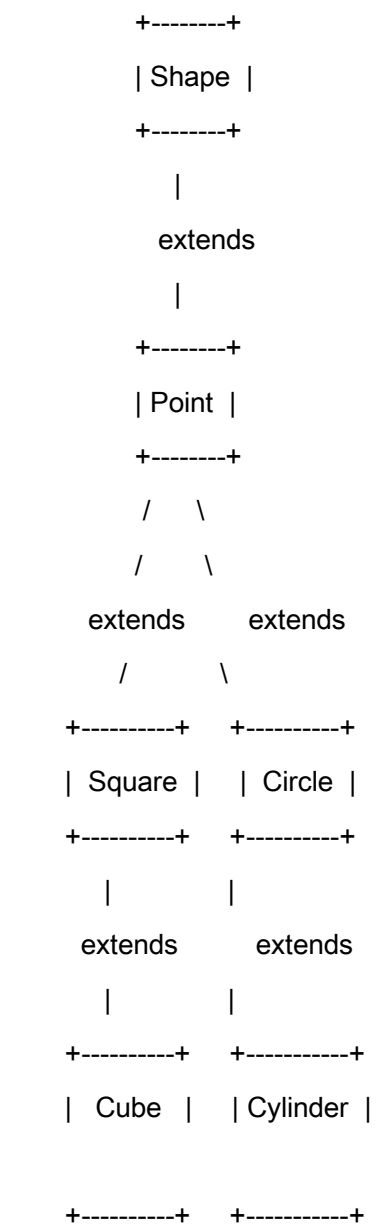
In Driver.java:

• Create one object of each:

- Point
- Square
- Cube
- Circle
- Cylinder

- Print each object
- Call area() where appropriate
- Call volume() where appropriate
- Output must be clean and readable

UML Diagram 1 — Inheritance Tree



Grading Rubric (30 Points)

Header Comment Blocks (4 pts)

- Present in all files
- Correct format
- Correct file names
- Proper description

Inheritance Structure (8 pts)

- Correct extends usage
- Proper use of super(...)

Fields & Constructors (6 pts)

- No duplicate inherited fields
- Correct parameter order
- Proper initialization

Methods (8 pts)

• area() correct
• volume() correct
• toString() overridden properly

Driver (4 pts)

• All required objects created
• Methods called correctly
• Output readable

Common Mistakes

- Forgetting super(...)
- Re-declaring x, y, z in child classes
- Incorrect constructor parameter order
- Using 3.14 instead of Math.PI
- Missing header comment block
- Poor indentation
- Submitting only some of the required files